PYTHON PROGRAMMING

Assessment Problem Statements

Comprehensive Coverage: Variables · Data Types · Operators · Control Flow
Lists · Tuples · Sets · Dictionaries · Strings · Functions
File Handling · Iterators · Generators · OOP · Stacks

Total Participants: 44
Organization: Wipro Technologies

Problem Statement Allocation

Each participant is assigned a unique problem statement as listed in the table below.

#	Name	Email ID	Problem Title
1	Shaik Hussain Vali	shaik.vali@wipro.com	Student Grade Management System
2	CHETHAN KUMAR.S.N	chethan.kumarsn@wipro.com	Bank Account Transaction System
3	Vignesh Varun B	vignesh.varunb@wipro.com	Library Book Management System
4	Kumar Kaushik	kumar.kaushik@wipro.com	Employee Payroll Processing System
5	Kajal Sunil Karad	kajal.karad@wipro.com	E-Commerce Shopping Cart System
6	Samjith T S	samjith.ts@wipro.com	Hospital Patient Record System
7	Bhagyashri Dhananjay Dhande	bhagyashri.dhande@wipro.com	Inventory Management System for a Warehouse
8	YASH VAIBHAV PAGDHARE	yash.pagdhare@wipro.com	Railway Ticket Reservation System
9	Athang Amit Patel	athang.patel@wipro.com	Online Quiz & Examination System
10	Tejas Sambhaji Dhole	tejas.dhole@wipro.com	Weather Monitoring & Alert System
11	Ramkisan Vijay Modokar	ramkisan.modokar@wipro.com	Social Media Post Scheduler
12	Mude Lokesh Naik	mudelokesh.naik@wipro.com	Electricity Bill Management System
13	Anshuman Sharma	anshuman.sharma@wipro.com	Flight Booking & Management System
14	Medipakala Brahma saroj	brahmasaroj.medipakala@wipro.com	Restaurant Order Management System
15	CHINTAPALLI CHAITANYA MADHAV	chintapallichaitanya.madhav@wipro.com	Vehicle Parking Management System
16	Renukuntla Gaurav	renukuntla.gaurav@wipro.com	Cricket Tournament Score Tracker
17	Puttoju Shishira	puttoju.shishira@wipro.com	Digital Voting & Election System
18	Bisam Pradeep Kumar	bisampradeep.kumar@wipro.com	Expense Tracker & Budget Planner
19	Gedela Haswanth	haswanth.gedela@wipro.com	University Course Registration System
20	Vemula Charishma	vemula.charishma@wipro.com	Smart Home Automation Controller

#	Name	Email ID	Problem Title
21	Palam Madhu Charan Reddy	palam.reddy@wipro.com	Movie Streaming Service Platform
22	DONTHI REDDY MEGHANA	donthireddy.meghana@wipro.com	Job Portal & Recruitment System
23	Sakhamuri Jayapaul Naidu	sakhamuri.jayapaulnaidu@wipro.com	Pharmacy Drug Inventory System
24	THOTA SIVA SAI SRIHARSHA	thota.sivasaisriharsha@wipro.com	Text-Based Role Playing Game Engine
25	SHAHIN AHMED F	shahinahmed.f@wipro.com	Supply Chain Tracking System
26	RUGVED SHRIKANT AGASTI	rugved.agasti@wipro.com	Email Filtering & Spam Classifier
27	JUJJUVARAPU VAMSI KRISHNA	vamsikrishna.jujjuvarapu@wipro.com	Customer Support Ticketing System
28	Suryaganesh Kannan	suryaganesh.k@wipro.com	Cryptocurrency Portfolio Tracker
29	JATHIKARTHA HANSHITH	jathikartha.hanshith@wipro.com	Agriculture Crop Yield Predictor
30	Mahankali kiran kumar	mahankali.kirankumar@wipro.com	News Feed Aggregator & Summarizer
31	Kalakuntla Sunil Gopi	kalakuntla.sunilgopi@wipro.com	Employee Attendance & Leave Management
32	Gandra Abhinaya	gandra.abhinaya@wipro.com	Food Delivery Order Tracker
33	Sathyamoorthy J	sathyamoorthy.j@wipro.com	Event Management & Ticketing System
34	SIMRITI R	simriti.r@wipro.com	Online Auction & Bidding Platform
35	Ankita Basak	ankita.basak1@wipro.com	Fitness Activity & Calorie Tracker
36	GARLAPATI VENKATA KRISHNA CHAITHANYA	krishnachaithanya.garlapati@wipro.com	Real Estate Property Listing System
37	NANDIMANDALAM SAKETH REDDY	nandimandalam.reddy@wipro.com	Compiler Lexical Token Analyzer
38	SURAPANENI POOJITHA	poojitha.surapaneni@wipro.com	Ride Sharing & Cab Booking System
39	Sabarish B	sabarish.b@wipro.com	Insurance Policy Management System
40	BALANIVETHA MURUGAN	balanivetha.murugan@wipro.com	Court Case Management System
41	CHADA VIJAY KANTH REDDY	chada.reddy@wipro.com	Scientific Calculator with History
42	R DRAVID	r.dravid@wipro.com	Smart Grid Energy Monitoring System
43	Madas Rushikesh	rushikesh.madas@wipro.com	Drone Fleet Management System

#	Name	Email ID	Problem Title
44	ALAHARI VARSHA	alahari.varsha@wipro.com	Cybersecurity Incident Response Tracker

Concepts Covered in All Problem Statements

Every problem statement is designed to holistically cover the following Python programming concepts:

Foundations

- Variables & Data Types
- Literals (int/float/bool/str/complex)
- Base Conversions (bin/oct/hex)
- Type Conversions (implicit & explicit)
- Arithmetic Operators
- Bitwise Operators
- Logical Operators
- Comparison Operators

Control Flow

- if Statement
- if-else Statement
- if-elif-else Statement
- for Loop
- while Loop
- break Statement
- continue Statement
- pass Statement
- match-case (Structural Pattern Matching)

Data Structures

- Lists (all built-in methods)
- List Comprehension (simple & nested)
- Tuples (immutable records)
- Sets (all set operations)
- Dictionaries (all dict methods)
- Dictionary Comprehension

Strings

- All 30+ string methods
- String formatting (format(), f-strings)
- Encoding & Translation methods

Functions

- Modular/Decomposed Design
- Positional-Only Parameters (/)
- Keyword-Only Parameters (*)
- *args and **kwargs
- Mixed Parameter Types
- Default Arguments
- Exception Handling (try/except/else/finally)

- Custom Exception Classes

File Handling

- Reading Files
- Writing Files
- Appending to Files
- with Statement (Context Manager)
- Error Handling (FileNotFoundError, IOError)

Advanced Python

- Custom Iterators (__iter__ / __next__)
- Generators (yield)
- Generator Expressions

OOP

- Classes & Objects
- Inheritance (Multi-level)
- __init__, __str__, __repr__
- Encapsulation (Private attributes)
- @property
- @classmethod, @staticmethod
- Method Overriding
- Polymorphism
- Operator Overloading
- isinstance() / issubclass()

Stacks

- Array-Based Stack (push/pop/peek/is_empty)
- Linked-List-Based Stack (Node + LinkedStack)
- Stack Underflow Handling
- Applied use: Undo/Redo, Audit Log, Navigation History

Problem Statement #01

Participant Name	Shaik Hussain Vali
Email ID	shaik.vali@wipro.com
Problem Title	**Student Grade Management System**
Domain	Education

Problem Description

You are required to build a Student Grade Management System in Python that covers all the following concepts:

Variables, Data Types & Literals:

- Store student name (string), roll number (int), GPA (float), and active status (bool).
- Use binary literal for permission flags (e.g., 0b1010) and hex literal for student ID.

Base Conversions & Type Conversions:

- Accept a decimal grade score, convert and display it in binary, octal, and hexadecimal.
- Explicitly convert string inputs to int/float where required.

Arithmetic & Bitwise Operators:

- Compute total marks, percentage, and grade using arithmetic operators.
- Use bitwise AND (&) to check if a student has both theory and lab credits set in a flag variable.

Logical & Comparison Operators:

- Use logical operators to check pass/fail conditions (marks > 40 and attendance > 75).
- Use comparison operators to rank students.

Conditional Statements (if / if-else / if-elif-else):

- Assign letter grades (A+, A, B, C, D, F) based on percentage using if-elif-else.
- Display scholarship eligibility if GPA > 9.0 and no backlogs.

Lists (with Functions & List Comprehension):

- Maintain a list of subject marks; use append(), remove(), sort(), reverse(), index(), count(), pop(), extend(), and copy().
- Use list comprehension to extract subjects where marks > 60.
- Use list comprehension with conditionals to flag failed subjects.

Tuples:

- Store immutable semester info as a tuple (semester_number, max_marks, passing_marks).
- Demonstrate tuple unpacking and the count()/index() methods.

Sets:

- Store elective subjects as a set. Demonstrate add(), discard(), union(), intersection(), difference(), and issubset().

Dictionaries:

- Use a dictionary to map subject names to marks.
- Demonstrate get(), keys(), values(), items(), update(), pop(), and dictionary comprehension to normalize marks.

for / while / break / continue / pass:

- Iterate over subjects with for loop; use continue to skip null entries and break on first failure found.
- Use while loop to re-enter marks until valid input.
- Use pass in a placeholder for a pending analysis function.

match-case:

- Use match-case to display performance category based on letter grade.

String Methods:

- Use upper(), lower(), strip(), replace(), split(), join(), find(), startswith(), endswith(), format(), zfill(), center(), title() on student name and remarks.

Functions (Modular Approach):

- calculate_percentage(marks: list, /, max_marks: int = 100) - positional-only.
- assign_grade(percentage, *, honors=False) - keyword-only argument for honors.
- generate_report(name, roll, *subjects, result="Pass", **extra_info) - mix of positional, *args, keyword, **kwargs.
- Exception handling using try/except to catch ValueError during mark entry and ZeroDivisionError during percentage calculation.

File Handling:

- Write the student report to a text file (student_report.txt).
- Read and display it back; use with statement and handle FileNotFoundError.

Iterators & Generators:

- Implement a custom iterator class SubjectIterator that iterates through subject-mark pairs.
- Write a generator function grade_generator(marks_list) that yields letter grades one by one.

OOP Concepts:

- Create classes: Person (base) → Student (derived) → GraduateStudent.
- Demonstrate __init__, __str__, __repr__, encapsulation (private __gpa), @property, @staticmethod, @classmethod, method overriding, and polymorphism.

Stacks (Array & Linked List):

- Implement a Stack using a Python list to undo the last entered mark (push/pop/peek/is_empty).
- Implement a Stack using a custom linked list (Node class) for storing the history of grade changes.

Problem Statement #02

Participant Name	CHETHAN KUMAR.S.N
Email ID	chethan.kumarsn@wipro.com
Problem Title	**Bank Account Transaction System**
Domain	Finance

Problem Description

Build a Bank Account Transaction System in Python covering all the following concepts:

Variables, Data Types & Literals:

- Store account_number (int, hex literal), holder_name (str), balance (float), is_active (bool).
- Use underscore separators in numeric literals (e.g., 1_000_000).

Base Conversions & Type Conversions:

- Display account number in binary and hexadecimal.
- Convert string amount inputs to float; convert transaction ID to int from hex string.

Arithmetic & Bitwise Operators:

- Compute interest, tax, and net balance using arithmetic operators.
- Use bitwise OR (|) to combine permission flags for account types.

Logical & Comparison Operators:

- Validate transaction eligibility using logical and comparison operators.

if / if-else / if-elif-else:

- Classify account type (Savings/Current/Premium) and apply corresponding interest rate.
- Check withdrawal limits using nested conditions.

Lists (Functions & Comprehension):

- Maintain transaction history as list of tuples; use sort(), reverse(), append(), pop(), extend(), index(), count(), clear(), copy(), insert().
- List comprehension to filter only debit transactions greater than a given threshold.

Tuples:

- Store account metadata (account_number, ifsc_code, branch_name) as a named-like tuple.
- Tuple unpacking and slicing.

Sets:

- Track unique ATM locations used; use union(), intersection(), difference(), symmetric_difference(), add(), discard(), frozenset.

Dictionaries:

- Map month names to total spending; use get(), setdefault(), update(), pop(), popitem(), items(), keys(), values(), and dict comprehension.

for / while / break / continue / pass:

- Iterate transaction list; use continue to skip cancelled transactions; break on suspicious activity.
- while loop for menu-driven transaction entry.
- pass in a pending fraud-detection function stub.

match-case:

- Use match-case to route to deposit, withdrawal, transfer, or statement display.

String Methods:

- Format account statements: ljust(), rjust(), center(), zfill(), format(), f-strings, upper(), lower(), strip(), replace(), split(), join(), startswith(), endswith(), find(), count().

Functions (Modular Approach):

- deposit(account, amount, /) - positional-only.
- calculate_interest(principal, *, rate, years) - keyword-only.
- generate_statement(holder, account_no, *transactions, currency="INR", **metadata) - mixed.
- Exception handling: try/except for ValueError, InsufficientFundsError (custom exception), and TypeError.

File Handling:

- Append each transaction to transactions.log; read and display last 5 transactions.
- Handle FileNotFoundError and PermissionError.

Iterators & Generators:

- Custom iterator class TransactionIterator for iterating over a filtered transaction list.
- Generator function monthly_summary(transactions) that yields (month, total) tuples.

OOP Concepts:

- BankAccount (base) → SavingsAccount, CurrentAccount, PremiumAccount (derived).
- Encapsulation (__balance), @property, @classmethod for factory methods, @staticmethod for IFSC validation, __str__, __repr__, operator overloading (__add__ for merging balances), polymorphism through overridden withdraw().

Stacks:

- Array-based stack to implement undo last transaction.
- Linked-list-based stack to maintain browser-style back navigation through account menu history.

Problem Statement #03

Participant Name	Vignesh Varun B
Email ID	vignesh.varunb@wipro.com
Problem Title	**Library Book Management System**
Domain	Library

Problem Description

Develop a Library Book Management System in Python covering all the following concepts:

Variables, Data Types & Literals:

- Store book_id (int, octal literal), title (str), price (float), available (bool).
- Use complex data type to represent a dummy ISBN-like coordinate.

Base Conversions & Type Conversions:

- Convert book_id to binary, octal, and hex; display all three.
- Convert string price input to float; int(price) for discount computation.

Arithmetic & Bitwise Operators:

- Calculate discounted price, fine per day, and total cost.
- Use bitwise left shift (<<) to double the fine for premium members.

Logical & Comparison Operators:

- Check if a book is overdue and fine applies using logical and comparison operators.

if / if-else / if-elif-else:

- Categorize book into genre (Fiction, Non-Fiction, Reference, Academic) using if-elif-else.
- Membership type check for borrow limit.

Lists (Functions & Comprehension):

- Maintain issued_books list; use all major list methods: append, insert, remove, pop, sort, reverse, index, count, clear, extend, copy.
- List comprehension to get books overdue by more than 7 days.
- Nested list comprehension for a 2D shelf layout.

Tuples:

- Store book metadata (isbn, publisher, year) as a tuple; use tuple methods count() and index().
- Demonstrate slicing and unpacking.

Sets:

- Track categories available in library; demonstrate set operations: union, intersection, difference, issubset, issuperset, symmetric_difference.

Dictionaries:

- Map member_id to list of borrowed books; use all dict methods.

- Dict comprehension to invert book_id → title mapping.

for / while / break / continue / pass:

- for loop to list overdue books; continue for returned books; break on first unavailable book.
- while loop for interactive search.
- pass in a future recommendation engine stub.

match-case:

- Route user action (search/borrow/return/renew/exit) using match-case.

String Methods:

- Use capitalize(), title(), upper(), lower(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), center(), count(), encode().

Functions:

- borrow_book(member_id, book_id, /) - positional-only.
- calculate_fine(days_overdue, *, rate_per_day=2.0) - keyword-only.
- generate_receipt(member, *books, discount=0, **extra) - mixed args.
- try/except: ValueError (invalid days input), KeyError (book not found), custom BookUnavailableError.

File Handling:

- Write/read a borrow_log.csv file; use with statement, handle FileNotFoundError and IOError.

Iterators & Generators:

- Iterator class BookShelfIterator that traverses books shelf by shelf.
- Generator overdue_books(records) yielding overdue entries lazily.

OOP:

- LibraryItem (base) → Book → EBook, ReferenceBook.
- Encapsulation, @property for fine calculation, @classmethod, @staticmethod, __str__, __len__ overloading, polymorphism via display_info().

Stacks:

- Array-based stack for borrow/return undo functionality.
- Linked-list-based stack for storing navigation history in the library portal.

Problem Statement #04

Participant Name	Kumar Kaushik
Email ID	kumar.kaushik@wipro.com
Problem Title	**Employee Payroll Processing System**
Domain	HR / Payroll

Problem Description

Build an Employee Payroll Processing System in Python covering all the following concepts:

Variables, Data Types & Literals:

- Store emp_id (int, hex literal), name (str), basic_salary (float), is_permanent (bool).
- Use binary literal to represent department flags.

Base Conversions & Type Conversions:

- Convert emp_id to binary/octal/hex; display.
- Convert string salary input to float; convert float tax to int for display.

Arithmetic & Bitwise Operators:

- Compute HRA, DA, PF, gross salary, tax deduction, and net salary.
- Use bitwise XOR (^) to toggle holiday bonus flag.

Logical & Comparison Operators:

- Validate salary range and tax slab eligibility.

if / if-else / if-elif-else:

- Apply income tax slabs (0%, 5%, 10%, 20%, 30%) using if-elif-else.
- Determine appraisal percentage based on performance rating.

Lists (Functions & Comprehension):

- Maintain all employee records as list of dicts; use append, remove, sort (key=lambda), pop, index, count, extend, copy, clear, insert, reverse.
- List comprehension: employees earning above average salary.
- List comprehension with nested condition for senior employees.

Tuples:

- Store tax slab info as tuple of tuples; unpack in loops.

Sets:

- Track departments; demonstrate add, discard, union, intersection, difference, frozenset.

Dictionaries:

- Map emp_id to payslip dict; use get(), setdefault(), update(), popitem(), items(), values(), keys().
- Dict comprehension to map emp_id → net_salary.

for / while / break / continue / pass:

- for loop over employee list; continue for inactive employees; break on duplicate ID detection.
- while loop for data entry with validation.
- pass as placeholder for performance analytics.

match-case:

- Match department code (HR/IT/FINANCE/MARKETING/OPS) to department name and policy.

String Methods:

- Format payslip header and footer using rjust(), ljust(), center(), zfill(), format(), f-strings; use upper(), lower(), strip(), replace(), split(), join(), find(), startswith(), endswith(), count().

Functions:

- compute_gross(basic, /, hra_pct=0.4, da_pct=0.1) - positional-only + defaults.
- apply_tax(gross, *, slab) - keyword-only.
- generate_payslip(emp_name, emp_id, *allowances, currency="INR", **deductions) - mixed.
- Exception handling: try/except for ValueError (invalid salary), ZeroDivisionError, custom InvalidDepartmentError.

File Handling:

- Write payslips to payroll_<month>.txt; read and display; use with, handle FileNotFoundError.

Iterators & Generators:

- Iterator class PayrollIterator for iterating over payslips in batches.
- Generator high_earners(employees, threshold) yielding employees above threshold.

OOP:

- Employee (base) → PermanentEmployee, ContractEmployee, Intern.
- Encapsulation (__salary), @property, @classmethod, @staticmethod, __str__, __repr__, comparison overloading (__lt__, __gt__ by salary), polymorphism via calculate_bonus().

Stacks:

- Array-based stack for undo salary correction.
- Linked-list-based stack for tracking audit log of payroll changes.

Problem Statement #05

Participant Name	Kajal Sunil Karad
Email ID	kajal.karad@wipro.com
Problem Title	**E-Commerce Shopping Cart System**
Domain	E-Commerce

Problem Description

Implement an E-Commerce Shopping Cart System in Python covering all the following:

Variables, Data Types & Literals:

- product_id (int, hex literal), product_name (str), price (float), in_stock (bool).
- Quantity using integer literal with underscore (10_000 units).

Base Conversions & Type Conversions:

- Convert product_id to binary and hex for barcode generation display.
- Convert string price input to float; convert discount_percentage str to int.

Arithmetic & Bitwise Operators:

- Compute subtotal, GST, discount, and final price.
- Use bitwise AND to check if a product has both 'express_delivery' and 'gift_wrap' options enabled via flags.

Logical & Comparison Operators:

- Validate cart rules: minimum order amount and maximum item count.

if / if-else / if-elif-else:

- Apply discount tiers (0%, 5%, 10%, 15%, 20%) based on cart total.
- Determine shipping tier (Free, Standard, Express).

Lists (Functions & Comprehension):

- Maintain cart as list; use all list methods including sort(key=), min(), max(), sum() with list.
- List comprehension for items above a price threshold.
- Nested list comprehension for a 2D product grid display.

Tuples:

- Store product metadata (sku, category, weight_kg) as tuple; unpack and use.

Sets:

- Track applied coupon codes; use add, discard, union, intersection to prevent duplicate coupon stacking.

Dictionaries:

- Inventory dict mapping product_id → {name, price, stock}; use all dict methods.
- Dict comprehension to create a price lookup.

for / while / break / continue / pass:

- for loop for cart summary; continue for out-of-stock items; break on payment success.
- while loop for quantity selection with validation.
- pass placeholder for recommendation engine.

match-case:

- Route to add_to_cart, remove_from_cart, view_cart, checkout, or exit.

String Methods:

- Format product listings; use ljust, rjust, center, format, f-string, upper, lower, title, strip, replace, split, join, find, startswith, endswith, zfill.

Functions:

- add_to_cart(cart, product_id, /) - positional-only.
- apply_coupon(cart_total, *, coupon_code) - keyword-only.
- checkout(customer_name, *cart_items, payment_method="UPI", **delivery_options) - mixed.
- try/except for ValueError (invalid qty), KeyError (missing product), custom OutOfStockError.

File Handling:

- Save order to order_<order_id>.txt; read recent orders; handle FileNotFoundError.

Iterators & Generators:

- Iterator class CartIterator for stepping through cart items.
- Generator discount_offers(products) yielding eligible offers.

OOP:

- Product (base) → PhysicalProduct, DigitalProduct, SubscriptionProduct.
- Encapsulation, @property, @classmethod for factory, @staticmethod for GST calculation, __str__, operator overloading __add__ (merge carts), polymorphism via calculate_shipping().

Stacks:

- Array-based stack for undo last cart action.
- Linked-list-based stack for browsing history (recently viewed products).

Problem Statement #06

Participant Name	Samjith T S
Email ID	samjith.ts@wipro.com
Problem Title	**Hospital Patient Record System**
Domain	Healthcare

Problem Description

Build a Hospital Patient Record System in Python covering all the following:

Variables, Data Types & Literals:

- patient_id (int, octal literal), name (str), temperature (float), is_admitted (bool).
- Blood pressure as complex number for systolic/diastolic representation.

Base Conversions & Type Conversions:

- Convert patient_id to binary and hex.
- Convert string vitals input to float; explicit int() for age.

Arithmetic & Bitwise Operators:

- Compute BMI, daily medicine cost, and insurance claim amount.
- Use bitwise operators to encode/decode patient status flags (ICU, Oxygen, Surgery).

Logical & Comparison Operators:

- Check critical care eligibility: temperature > 103 or SpO2 < 90 and age > 60.

if / if-else / if-elif-else:

- Categorize BMI (Underweight, Normal, Overweight, Obese).
- Assign ward (General/Semi-Private/Private/ICU) based on condition.

Lists (Functions & Comprehension):

- Maintain prescription list; use all list methods.
- List comprehension: patients admitted for more than 5 days.
- Nested comprehension for a weekly vitals grid.

Tuples:

- Store patient vitals snapshot (temp, bp_sys, bp_dia, spo2, pulse) as tuple; unpack.

Sets:

- Track allergies as a set; use add, discard, union, intersection across patients.

Dictionaries:

- Map patient_id → record dict; use all dict methods.
- Dict comprehension for critical patients summary.

for / while / break / continue / pass:

- Iterate ward rooms; continue for empty beds; break on first available ICU.
- while loop for vitals monitoring until stable.
- pass in a future AI diagnosis stub.

match-case:

- Match blood group (A+/A-/B+/B-/AB+/AB-/O+/O-) to compatible donors list.

String Methods:

- Format patient ID cards and prescriptions; use all major string methods.

Functions:

- calculate_bmi(weight_kg, height_m, /) - positional-only.
- check_eligibility(age, *, insurance_type) - keyword-only.
- admit_patient(patient_name, patient_id, *symptoms, ward="General", **contact_info) - mixed.
- try/except: ValueError (invalid vitals), custom CriticalConditionError, FileNotFoundError.

File Handling:

- Append daily vitals to patient_<id>_vitals.txt; read and display history.

Iterators & Generators:

- Iterator class WardIterator for traversing patients ward-by-ward.
- Generator critical_alerts(patients) yielding patients needing urgent attention.

OOP:

- Person → Patient → InPatient, OutPatient; Doctor.
- Encapsulation (__medical_history), @property, @classmethod, @staticmethod, __str__, polymorphism via generate_report().

Stacks:

- Array-based stack for undo last prescription entry.
- Linked-list-based stack for tracking patient's consultation history.

Problem Statement #07

Participant Name	Bhagyashri Dhananjay Dhande
Email ID	bhagyashri.dhande@wipro.com
Problem Title	**Inventory Management System for a Warehouse**
Domain	Logistics

Problem Description

Develop an Inventory Management System for a Warehouse in Python covering all the following:

Variables, Data Types & Literals:

- item_code (int, hex literal), item_name (str), unit_price (float), is_perishable (bool).
- Reorder level using binary literal.

Base Conversions & Type Conversions:

- Display item_code in binary/octal/hex.
- Convert user input strings to int/float for quantity and price.

Arithmetic & Bitwise Operators:

- Compute total inventory value, reorder cost, and profit margin.
- Use bitwise shift (>>) to compute half-stock threshold.

Logical & Comparison Operators:

- Trigger reorder alert: quantity < reorder_level and supplier_active.

if / if-else / if-elif-else:

- Classify items as Fast Moving, Slow Moving, or Dead Stock.
- Apply different tax rates based on item category.

Lists (Functions & Comprehension):

- Maintain stock list; use append, insert, remove, pop, sort, reverse, index, count, extend, copy, clear.
- List comprehension: items below minimum stock.
- List comprehension with walrus operator (:=) to filter and capture value.

Tuples:

- Store supplier info as immutable tuple (supplier_id, name, lead_time_days).

Sets:

- Track categories in stock; set operations for category analysis.

Dictionaries:

- Map item_code → stock_dict; comprehensive use of all dict methods.
- Dict comprehension for reorder quantities.

for / while / break / continue / pass:

- for loop for stock audit; continue for non-perishables during expiry check; break on discrepancy.
- while loop for data entry.
- pass in a future demand forecasting stub.

match-case:

- Match warehouse zone (A/B/C/D) to storage conditions.

String Methods:

- Format stock reports and item labels using all string methods.

Functions:

- update_stock(item_code, quantity, /) - positional-only.
- calculate_reorder_qty(current_stock, *, safety_stock, lead_time_demand) - keyword-only.
- generate_stock_report(warehouse_id, *items, currency="INR", **metadata) - mixed.
- try/except: ValueError, KeyError (item not found), custom InsufficientStockError.

File Handling:

- Read/write inventory to inventory.csv; handle FileNotFoundError, csv.Error.

Iterators & Generators:

- Iterator class ZoneIterator for iterating items by warehouse zone.
- Generator low_stock_alerts(inventory) yielding items needing reorder.

OOP:

- Item (base) → PerishableItem, NonPerishableItem, ElectronicItem.
- Encapsulation, @property, @classmethod, @staticmethod, __str__, __repr__, __len__, polymorphism via compute_tax().

Stacks:

- Array-based stack for undo last stock update.
- Linked-list-based stack for transaction rollback history.

Problem Statement #08

Participant Name	YASH VAIBHAV PAGDHARE
Email ID	yash.pagdhare@wipro.com
Problem Title	**Railway Ticket Reservation System**
Domain	Transportation

Problem Description

Implement a Railway Ticket Reservation System in Python covering all the following:

Variables, Data Types & Literals:

- train_number (int, octal literal), route (str), fare (float), is_available (bool).
- Seat number using binary literal.

Base Conversions & Type Conversions:

- Convert train_number to binary and hex for display.
- Convert string date/fare inputs to int/float.

Arithmetic & Bitwise Operators:

- Compute total fare with GST, senior citizen discount, and tatkal charge.
- Use bitwise operators to manage seat availability bitmask (64-seat coach).

Logical & Comparison Operators:

- Check booking eligibility: seats_available > 0 and journey_date is valid.

if / if-else / if-elif-else:

- Determine coach type (Sleeper/3AC/2AC/1AC/Chair) and corresponding fare.
- Apply age-based concessions.

Lists (Functions & Comprehension):

- Maintain waitlist as a list; all major list operations.
- List comprehension: passengers travelling more than 500 km.

Tuples:

- Station info (station_code, station_name, arrival_time, departure_time) as tuple.

Sets:

- Track booked seat numbers; use set operations for available seat computation.

Dictionaries:

- Map PNR → booking details dict; all dict methods.
- Dict comprehension for train → available_seats mapping.

for / while / break / continue / pass:

- for loop for coach seat scan; continue for reserved seats; break on first available.
- while loop for seat selection with validation.
- pass for future dynamic pricing engine stub.

match-case:

- Match travel class to quota and concession rules.

String Methods:

- Format ticket display: PNR, name, seat, coach, journey details; use all string methods.

Functions:

- book_ticket(pnr, passenger_name, /) - positional-only.
- apply_concession(fare, *, category) - keyword-only.
- generate_ticket(train_no, *passengers, travel_class="SL", **journey_details) - mixed.
- try/except: ValueError, custom SeatNotAvailableError, custom WaitlistFullError.

File Handling:

- Write PNR receipts to tickets/PNR_<number>.txt; read on cancellation; handle exceptions.

Iterators & Generators:

- Iterator class CoachIterator traversing seats coach by coach.
- Generator waitlist_generator(waitlist) yielding confirmed passengers as cancellations occur.

OOP:

- Transport (base) → Train, Express, Rajdhani.
- Encapsulation, @property, @classmethod, @staticmethod (PNR generator), __str__, polymorphism via calculate_fare().

Stacks:

- Array-based stack for undo last booking.
- Linked-list-based stack for cancellation history.

Problem Statement #09

Participant Name	Athang Amit Patel
Email ID	athang.patel@wipro.com
Problem Title	**Online Quiz & Examination System**
Domain	EdTech

Problem Description

Build an Online Quiz & Examination System in Python covering all the following:

Variables, Data Types & Literals:

- question_id (int, binary literal), topic (str), marks (float), timed (bool).
- Negative marking factor as float literal (e.g., -0.25).

Base Conversions & Type Conversions:

- Convert question_id to octal and hex.
- Convert string time_limit input to int.

Arithmetic & Bitwise Operators:

- Compute total score with negative marking; time penalty calculation.
- Use bitwise OR to combine attempt flags across multiple sections.

Logical & Comparison Operators:

- Check pass condition: total_score >= pass_marks and attempt_count < max_attempts.

if / if-else / if-elif-else:

- Assign result grade (Distinction/First/Second/Pass/Fail) from score percentage.
- Determine topic-wise performance level.

Lists (Functions & Comprehension):

- Manage question bank list; all list methods.
- List comprehension: questions with difficulty > 3.
- Nested list comprehension for a topic-difficulty matrix.

Tuples:

- Store question data (q_id, question_text, options_tuple, correct_answer) as a nested tuple.

Sets:

- Track attempted question IDs; set operations to find unattempted questions.

Dictionaries:

- Map student_id → {score, time_taken, answers}; all dict methods.
- Dict comprehension for topic-wise average score.

for / while / break / continue / pass:

- for loop to present questions; continue on skipped; break when time runs out.
- while loop for answer validation retry.
- pass for future AI question generator stub.

match-case:

- Match exam type (MCQ/True-False/Fill-in-the-Blank/Subjective) to scoring logic.

String Methods:

- Format question paper and result card; use all string methods.

Functions:

- load_question(q_id, topic, /) - positional-only.
- evaluate_answer(response, *, correct, negative_marks=0.25) - keyword-only.
- generate_result_card(student_name, roll_no, *scores, grade_scale="10", **metadata) - mixed.
- try/except: ValueError, IndexError, custom TimeExceededError, custom MaxAttemptsError.

File Handling:

- Read questions from questions.txt; write results to results_<student_id>.txt.

Iterators & Generators:

- Iterator class QuestionIterator for section-wise question delivery.
- Generator random_question_picker(bank, n) yielding n random questions.

OOP:

- Question (base) → MCQ, TrueFalse, FillInTheBlank.
- ExamSession class with encapsulation, @property for score, @classmethod, @staticmethod (timer), polymorphism via check_answer().

Stacks:

- Array-based stack for back navigation between questions.
- Linked-list-based stack for tracking answer change history.

Problem Statement #10

Participant Name	Tejas Sambhaji Dhole
Email ID	tejas.dhole@wipro.com
Problem Title	**Weather Monitoring & Alert System**
Domain	Meteorology

Problem Description

Develop a Weather Monitoring & Alert System in Python covering all the following:

Variables, Data Types & Literals:

- station_id (int, hex literal), location (str), temperature (float), is_storm (bool).
- Wind speed in float literal; humidity as integer literal.

Base Conversions & Type Conversions:

- Display station_id in binary and octal.
- Convert Celsius to Fahrenheit and Kelvin explicitly using float conversions.

Arithmetic & Bitwise Operators:

- Compute heat index, wind chill, and dew point.
- Use bitwise flags to represent active alerts (rain|storm|flood|heatwave).

Logical & Comparison Operators:

- Trigger alerts: temp > 40 or humidity > 90 and wind_speed > 60.

if / if-else / if-elif-else:

- Classify weather (Sunny/Cloudy/Rainy/Stormy/Foggy/Snowy).
- Determine advisory level (Green/Yellow/Orange/Red).

Lists (Functions & Comprehension):

- Store hourly readings as list of dicts; all list methods.
- List comprehension: hours where temperature exceeded 38°C.

Tuples:

- Store station coordinates (lat, lon, elevation, timezone) as tuple; unpack.

Sets:

- Collect unique alert types generated; set operations across stations.

Dictionaries:

- Map station_id → weather_data dict; all dict methods.
- Dict comprehension for daily average computation.

for / while / break / continue / pass:

- for loop over hourly data; continue for missing readings; break on extreme event.
- while loop for real-time simulation.
- pass for future ML prediction stub.

match-case:

- Match alert level to corresponding safety instructions.

String Methods:

- Format weather bulletins and SMS alerts; use all string methods.

Functions:

- celsius_to_fahrenheit(temp_c, /) - positional-only.
- compute_heat_index(temp, *, humidity) - keyword-only.
- generate_alert(station_id, location, *readings, alert_type="General", **sensor_data) - mixed.
- try/except: ValueError, custom SensorFailureError, ZeroDivisionError.

File Handling:

- Log readings to weather_log_<date>.txt; read and compute daily stats.

Iterators & Generators:

- Iterator class StationDataIterator for iterating time-series data.
- Generator hourly_alerts(hourly_data) yielding hours with anomalies.

OOP:

- Sensor (base) → TemperatureSensor, RainfallSensor, WindSensor.
- WeatherStation class; encapsulation, @property, @classmethod, @staticmethod, polymorphism via generate_reading().

Stacks:

- Array-based stack for rolling window of last 10 readings.
- Linked-list-based stack for alert escalation history.

Problem Statement #11

Participant Name	Ramkisan Vijay Modokar
Email ID	ramkisan.modokar@wipro.com
Problem Title	**Social Media Post Scheduler**
Domain	Social Media

Problem Description

Build a Social Media Post Scheduler in Python covering all the following:

Variables, Data Types & Literals:

- post_id (int, binary literal), platform (str), reach_score (float), is_published (bool).
- Character limit using integer literal with underscore.

Base Conversions & Type Conversions:

- Display post_id in hex and octal; convert scheduled_time string to int.

Arithmetic & Bitwise Operators:

- Compute engagement rate, optimal posting time score.
- Use bitwise flags to encode platforms (Twitter|LinkedIn|Instagram|Facebook).

Logical & Comparison Operators:

- Validate post: content not empty and length <= char_limit and scheduled_time valid.

if / if-else / if-elif-else:

- Apply platform-specific formatting rules and posting restrictions.

Lists (Functions & Comprehension):

- Maintain post queue; all list methods.
- List comprehension: posts with engagement_rate > 5%.

Tuples:

- Store platform config (name, char_limit, max_hashtags, supports_images) as tuple.

Sets:

- Track used hashtags; set operations for trending tag analysis.

Dictionaries:

- Map platform → analytics dict; all dict methods.
- Dict comprehension for best performing platforms.

for / while / break / continue / pass:

- for loop over post queue; continue for draft posts; break on API failure.
- while loop for scheduling retry.

- pass for future auto-reply stub.

match-case:

- Match platform name to its API endpoint handler.

String Methods:

- Format post previews; use all string methods including encode(), decode(), maketrans().

Functions:

- schedule_post(post_id, content, /) - positional-only.
- set_auto_publish(post_id, *, scheduled_time) - keyword-only.
- create_campaign(*post_ids, platform="All", **campaign_settings) - mixed.
- try/except: ValueError, custom PostTooLongError, custom ScheduleConflictError.

File Handling:

- Write schedule to posts_schedule.json (as plain text); read and display upcoming posts.

Iterators & Generators:

- Iterator class PostQueueIterator for chronological post delivery.
- Generator optimal_slots(analytics) yielding best posting times.

OOP:

- Post (base) → TextPost, ImagePost, VideoPost, StoryPost.
- Scheduler class; encapsulation, @property, @classmethod, @staticmethod (hashtag extractor), polymorphism via publish().

Stacks:

- Array-based stack for undo scheduled post.
- Linked-list-based stack for posting history log.

Problem Statement #12

Participant Name	Mude Lokesh Naik
Email ID	mudelokesh.naik@wipro.com
Problem Title	**Electricity Bill Management System**
Domain	Utilities

Problem Description

Implement an Electricity Bill Management System in Python covering all the following:

Variables, Data Types & Literals:

- consumer_id (int, hex literal), name (str), units_consumed (float), is_commercial (bool).
- Tariff slab boundaries as integer literals.

Base Conversions & Type Conversions:

- Display consumer_id in binary and octal.
- Convert string units input to float; convert tax amount to int for display.

Arithmetic & Bitwise Operators:

- Compute slab-wise charges, surcharge, and net bill.
- Use bitwise AND to check if consumer has solar_subsidy and smart_meter flags enabled.

Logical & Comparison Operators:

- Apply late payment penalty: due_date_passed and balance_due > 0.

if / if-else / if-elif-else:

- Apply consumption slabs (0-100, 101-300, 301-500, >500 units) with different tariffs.
- Determine consumer category and applicable subsidies.

Lists (Functions & Comprehension):

- Maintain 12-month billing history list; all list methods.
- List comprehension: months with bill > average.

Tuples:

- Store meter reading snapshot (reading_date, previous, current, units) as tuple.

Sets:

- Track payment methods used; set operations for reporting.

Dictionaries:

- Map consumer_id → billing_record dict; all dict methods.
- Dict comprehension for monthly consumption summary.

for / while / break / continue / pass:

- Iterate billing cycles; continue for zero-consumption months; break on erroneous reading.
- while loop for payment processing.
- pass for future smart grid integration stub.

match-case:

- Match consumer_category (Domestic/Commercial/Industrial/Agricultural) to tariff.

String Methods:

- Format bill statements; use all string methods.

Functions:

- calculate_slab_charge(units, /) - positional-only.
- apply_subsidy(bill_amount, *, consumer_type) - keyword-only.
- generate_bill(consumer_name, consumer_id, *slab_charges, tax_rate=0.05, **extra) - mixed.
- try/except: ValueError, custom InvalidMeterReadingError, ZeroDivisionError.

File Handling:

- Write monthly bills to bills/consumer_<id>_<month>.txt; read payment history.

Iterators & Generators:

- Iterator class BillingCycleIterator for monthly bill iteration.
- Generator high_consumption_alerts(records, threshold) yielding consumers over threshold.

OOP:

- Consumer (base) → DomesticConsumer, CommercialConsumer, IndustrialConsumer.
- Encapsulation, @property, @classmethod, @staticmethod (unit validator), __str__, polymorphism via calculate_bill().

Stacks:

- Array-based stack for bill correction undo.
- Linked-list-based stack for payment transaction history.

Problem Statement #13

Participant Name	Anshuman Sharma
Email ID	anshuman.sharma@wipro.com
Problem Title	**Flight Booking & Management System**
Domain	Aviation

Problem Description

Build a Flight Booking & Management System in Python covering all the following:

Variables, Data Types & Literals:

- flight_id (int, octal literal), airline (str), base_fare (float), is_direct (bool).
- Max seats as integer literal with underscore.

Base Conversions & Type Conversions:

- Convert flight_id to binary and hex; display.
- Convert string input fares to float; type-cast seat numbers to int.

Arithmetic & Bitwise Operators:

- Compute final fare with fuel surcharge, airport tax, and convenience fee.
- Use bitwise flags for service inclusions (meal|baggage|lounge|priority_boarding).

Logical & Comparison Operators:

- Validate booking: seats_available > 0 and departure_time is future and passenger_age >= 0.

if / if-else / if-elif-else:

- Determine ticket class (Economy/Premium Economy/Business/First) and associated perks.
- Apply infant/child/senior concessions.

Lists (Functions & Comprehension):

- Maintain passenger manifest list; all list methods.
- List comprehension: passengers with more than 20 kg baggage.

Tuples:

- Store flight schedule (dep_airport, arr_airport, dep_time, arr_time, duration) as tuple.

Sets:

- Track booked seat numbers; set operations for available seats.

Dictionaries:

- Map PNR → booking dict; all dict methods.
- Dict comprehension for origin-destination pair fare mapping.

for / while / break / continue / pass:

- for loop seat assignment; continue for blocked seats; break on emergency exit row constraint.
- while loop for fare class selection.
- pass for future dynamic pricing stub.

match-case:

- Match service flag combination to boarding category.

String Methods:

- Format boarding pass and itinerary; use all string methods.

Functions:

- book_seat(pnr, passenger_name, /) - positional-only.
- apply_fare_rule(base_fare, *, class_multiplier) - keyword-only.
- generate_boarding_pass(passenger_name, flight_id, *segments, seat="Economy", **extras) - mixed.
- try/except: ValueError, custom SeatOccupiedError, custom FlightCancelledError.

File Handling:

- Write boarding passes to boarding_passes/<PNR>.txt; read on check-in.

Iterators & Generators:

- Iterator class ManifestIterator for boarding group-wise passenger iteration.
- Generator fare_calendar(dates, route) yielding (date, fare) pairs.

OOP:

- Aircraft (base) → NarrowBody, WideBody.
- Booking class; encapsulation, @property, @classmethod, @staticmethod (seat map generator), polymorphism via compute_fare().

Stacks:

- Array-based stack for undoing seat selection.
- Linked-list-based stack for flight search history.

Problem Statement #14

Participant Name	Medipakala Brahma saroj
Email ID	brahmasaroj.medipakala@wipro.com
Problem Title	**Restaurant Order Management System**
Domain	Food & Hospitality

Problem Description

Implement a Restaurant Order Management System in Python covering all the following:

Variables, Data Types & Literals:

- order_id (int, binary literal), table_number (int), total_amount (float), is_veg (bool).
- Tax rate as float literal; waiting time as int literal.

Base Conversions & Type Conversions:

- Display order_id in hex and octal.
- Convert string price inputs to float; explicit int for quantity.

Arithmetic & Bitwise Operators:

- Compute subtotal, GST (5%/18%), service charge, and grand total.
- Use bitwise OR/AND to encode dietary flags (veg|vegan|gluten-free|nut-free).

Logical & Comparison Operators:

- Apply happy hour discount: current_hour >= 14 and current_hour <= 17 and order_total >= 500.

if / if-else / if-elif-else:

- Assign table section (Outdoor/Indoor/Private/Bar) based on group size and preference.
- Apply discount tiers for order amount.

Lists (Functions & Comprehension):

- Maintain menu and order item lists; all list methods.
- List comprehension: items with preparation time > 15 min.

Tuples:

- Store menu item details (item_id, name, price, category, prep_time_mins) as tuple.

Sets:

- Track active table numbers; set operations for available/occupied tables.

Dictionaries:

- Map order_id → order details dict; all dict methods.
- Dict comprehension for category-wise item count.

for / while / break / continue / pass:

- for loop over order items; continue for unavailable items; break on kitchen closure.
- while loop for order modification.
- pass for future automated reorder stub.

match-case:

- Match payment_method (Cash/Card/UPI/Wallet/Voucher) to processing logic.

String Methods:

- Format KOT (Kitchen Order Ticket) and bill; use all string methods.

Functions:

- add_item(order_id, item_id, /) - positional-only.
- apply_discount(total, *, membership_level) - keyword-only.
- generate_bill(table_no, *order_items, gst_rate=0.05, **payment_details) - mixed.
- try/except: ValueError, custom ItemUnavailableError, custom TableOccupiedError.

File Handling:

- Write KOT to kot/<order_id>.txt; append to daily_sales.txt.

Iterators & Generators:

- Iterator class MenuIterator for category-wise menu traversal.
- Generator daily_specials(menu) yielding today's special items.

OOP:

- MenuItem (base) → Starter, MainCourse, Dessert, Beverage.
- Order class; encapsulation, @property, @classmethod, @staticmethod (tax calculator), __str__, polymorphism via prepare().

Stacks:

- Array-based stack for undo last item addition to order.
- Linked-list-based stack for order modification history.

Problem Statement #15

Participant Name	CHINTAPALLI CHAITANYA MADHAV
Email ID	chintapallichaitanya.madhav@wipro.com
Problem Title	**Vehicle Parking Management System**
Domain	Smart City

Problem Description

Build a Vehicle Parking Management System in Python covering all the following:

Variables, Data Types & Literals:

- slot_id (int, hex literal), vehicle_number (str), hourly_rate (float), is_occupied (bool).
- Total capacity as integer literal with underscore.

Base Conversions & Type Conversions:

- Convert slot_id to binary and octal.
- Convert string entry/exit time input to int for duration calculation.

Arithmetic & Bitwise Operators:

- Compute parking fee based on duration; apply overnight/weekend surcharge.
- Use bitwise flags to track slot types (two-wheeler|four-wheeler|EV|handicapped).

Logical & Comparison Operators:

- Check slot eligibility: slot_type matches vehicle_type and not is_occupied.

if / if-else / if-elif-else:

- Apply tier-based rates (first 2 hrs, 2-6 hrs, 6-12 hrs, 12-24 hrs, >24 hrs).
- Determine fine for overstay.

Lists (Functions & Comprehension):

- Maintain list of all parking records; all list methods.
- List comprehension: vehicles parked for more than 6 hours.

Tuples:

- Store parking record (vehicle_no, entry_time, slot_id, vehicle_type) as tuple.

Sets:

- Track occupied slot IDs; set operations for available slots by type.

Dictionaries:

- Map vehicle_number → parking_record; all dict methods.
- Dict comprehension for slot-type occupancy summary.

for / while / break / continue / pass:

- for loop over floor slots; continue for reserved slots; break on first available.
- while loop for payment processing retry.
- pass for future license plate recognition stub.

match-case:

- Match vehicle_type (Bike/Car/EV/Bus/Truck) to slot category and rate.

String Methods:

- Format parking ticket and receipt; use all string methods.

Functions:

- allocate_slot(vehicle_no, vehicle_type, /) - positional-only.
- compute_fee(duration_hours, *, vehicle_type, is_member=False) - keyword-only.
- generate_receipt(vehicle_no, slot_id, *durations, payment_mode="Cash", **extras) - mixed.
- try/except: ValueError, custom SlotNotAvailableError, custom InvalidVehicleTypeError.

File Handling:

- Log entries/exits to parking_log_<date>.txt; read for revenue report.

Iterators & Generators:

- Iterator class FloorIterator for floor-by-floor slot traversal.
- Generator revenue_generator(log_file_content) yielding daily revenue entries.

OOP:

- Vehicle (base) → TwoWheeler, FourWheeler, EV, HeavyVehicle.
- ParkingSlot and ParkingSystem classes; encapsulation, @property, @classmethod, @staticmethod (slot ID generator), polymorphism via compute_charge().

Stacks:

- Array-based stack for undo last slot allocation.
- Linked-list-based stack for vehicle entry/exit event history.

Problem Statement #16

Participant Name	Renukuntla Gaurav
Email ID	renukuntla.gaurav@wipro.com
Problem Title	**Cricket Tournament Score Tracker**
Domain	Sports

Problem Description

Build a Cricket Tournament Score Tracker in Python (Sports domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Sports domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #17

Participant Name	Puttoju Shishira
Email ID	puttoju.shishira@wipro.com
Problem Title	**Digital Voting & Election System**
Domain	Governance

Problem Description

Build a Digital Voting & Election System in Python (Governance domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Governance domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #18

Participant Name	Bisam Pradeep Kumar
Email ID	bisampradeep.kumar@wipro.com
Problem Title	**Expense Tracker & Budget Planner**
Domain	Personal Finance

Problem Description

Build a Expense Tracker & Budget Planner in Python (Personal Finance domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Personal Finance domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #19

Participant Name	Gedela Haswanth
Email ID	haswanth.gedela@wipro.com
Problem Title	**University Course Registration System**
Domain	Academia

Problem Description

Build a University Course Registration System in Python (Academia domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Academia domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #20

Participant Name	Vemula Charishma
Email ID	vemula.charishma@wipro.com
Problem Title	**Smart Home Automation Controller**
Domain	IoT

Problem Description

Build a Smart Home Automation Controller in Python (IoT domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the IoT domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #21

Participant Name	Palam Madhu Charan Reddy
Email ID	palam.reddy@wipro.com
Problem Title	**Movie Streaming Service Platform**
Domain	Entertainment

Problem Description

Build a Movie Streaming Service Platform in Python (Entertainment domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Entertainment domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #22

Participant Name	DONTHI REDDY MEGHANA
Email ID	donthireddy.meghana@wipro.com
Problem Title	**Job Portal & Recruitment System**
Domain	HR Tech

Problem Description

Build a Job Portal & Recruitment System in Python (HR Tech domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the HR Tech domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #23

Participant Name	Sakhamuri Jayapaul Naidu
Email ID	sakhamuri.jayapaulnaidu@wipro.com
Problem Title	**Pharmacy Drug Inventory System**
Domain	Pharma

Problem Description

Build a Pharmacy Drug Inventory System in Python (Pharma domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Pharma domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #24

Participant Name	THOTA SIVA SAI SRIHARSHA
Email ID	thota.sivasaisriharsha@wipro.com
Problem Title	**Text-Based Role Playing Game Engine**
Domain	Gaming

Problem Description

Build a Text-Based Role Playing Game Engine in Python (Gaming domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Gaming domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #25

Participant Name	SHAHIN AHMED F
Email ID	shahinahmed.f@wipro.com
Problem Title	**Supply Chain Tracking System**
Domain	Logistics

Problem Description

Build a Supply Chain Tracking System in Python (Logistics domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Logistics domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #26

Participant Name	RUGVED SHRIKANT AGASTI
Email ID	rugved.agasti@wipro.com
Problem Title	**Email Filtering & Spam Classifier**
Domain	Cybersecurity

Problem Description

Build a Email Filtering & Spam Classifier in Python (Cybersecurity domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Cybersecurity domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #27

Participant Name	JUJJUVARAPU VAMSI KRISHNA
Email ID	vamsikrishna.jujjuvarapu@wipro.com
Problem Title	**Customer Support Ticketing System**
Domain	IT Services

Problem Description

Build a Customer Support Ticketing System in Python (IT Services domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the IT Services domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #28

Participant Name	Suryaganesh Kannan
Email ID	suryaganesh.k@wipro.com
Problem Title	**Cryptocurrency Portfolio Tracker**
Domain	FinTech

Problem Description

Build a Cryptocurrency Portfolio Tracker in Python (FinTech domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the FinTech domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #29

Participant Name	JATHIKARTHA HANSHITH
Email ID	jathikartha.hanshith@wipro.com
Problem Title	**Agriculture Crop Yield Predictor**
Domain	AgriTech

Problem Description

Build a Agriculture Crop Yield Predictor in Python (AgriTech domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the AgriTech domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #30

Participant Name	Mahankali kiran kumar
Email ID	mahankali.kirankumar@wipro.com
Problem Title	**News Feed Aggregator & Summarizer**
Domain	Media

Problem Description

Build a News Feed Aggregator & Summarizer in Python (Media domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Media domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #31

Participant Name	Kalakuntla Sunil Gopi
Email ID	kalakuntla.sunilgopi@wipro.com
Problem Title	**Employee Attendance & Leave Management**
Domain	HR

Problem Description

Build a Employee Attendance & Leave Management in Python (HR domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the HR domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #32

Participant Name	Gandra Abhinaya
Email ID	gandra.abhinaya@wipro.com
Problem Title	**Food Delivery Order Tracker**
Domain	Food Tech

Problem Description

Build a Food Delivery Order Tracker in Python (Food Tech domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Food Tech domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #33

Participant Name	Sathyamoorthy J
Email ID	sathyamoorthy.j@wipro.com
Problem Title	**Event Management & Ticketing System**
Domain	Events

Problem Description

Build a Event Management & Ticketing System in Python (Events domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Events domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #34

Participant Name	SIMRITI R
Email ID	simriti.r@wipro.com
Problem Title	**Online Auction & Bidding Platform**
Domain	E-Commerce

Problem Description

Build a Online Auction & Bidding Platform in Python (E-Commerce domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the E-Commerce domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #35

Participant Name	Ankita Basak
Email ID	ankita.basak1@wipro.com
Problem Title	**Fitness Activity & Calorie Tracker**
Domain	Health & Wellness

Problem Description

Build a Fitness Activity & Calorie Tracker in Python (Health & Wellness domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Health & Wellness domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #36

Participant Name	GARLAPATI VENKATA KRISHNA CHAITHANYA
Email ID	krishnachaithanya.garlapati@wipro.com
Problem Title	**Real Estate Property Listing System**
Domain	Real Estate

Problem Description

Build a Real Estate Property Listing System in Python (Real Estate domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Real Estate domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #37

Participant Name	NANDIMANDALAM SAKETH REDDY
Email ID	nandimandalam.reddy@wipro.com
Problem Title	**Compiler Lexical Token Analyzer**
Domain	Computer Science

Problem Description

Build a Compiler Lexical Token Analyzer in Python (Computer Science domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Computer Science domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #38

Participant Name	SURAPANENI POOJITHA
Email ID	poojitha.surapaneni@wipro.com
Problem Title	**Ride Sharing & Cab Booking System**
Domain	MaaS

Problem Description

Build a Ride Sharing & Cab Booking System in Python (MaaS domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the MaaS domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #39

Participant Name	Sabarish B
Email ID	sabarish.b@wipro.com
Problem Title	**Insurance Policy Management System**
Domain	Insurance

Problem Description

Build a Insurance Policy Management System in Python (Insurance domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Insurance domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #40

Participant Name	BALANIVETHA MURUGAN
Email ID	balanivetha.murugan@wipro.com
Problem Title	**Court Case Management System**
Domain	Legal

Problem Description

Build a Court Case Management System in Python (Legal domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Legal domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #41

Participant Name	CHADA VIJAY KANTH REDDY
Email ID	chada.reddy@wipro.com
Problem Title	**Scientific Calculator with History**
Domain	Utilities

Problem Description

Build a Scientific Calculator with History in Python (Utilities domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Utilities domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #42

Participant Name	R DRAVID
Email ID	r.dravid@wipro.com
Problem Title	**Smart Grid Energy Monitoring System**
Domain	Energy

Problem Description

Build a Smart Grid Energy Monitoring System in Python (Energy domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Energy domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #43

Participant Name	Madas Rushikesh
Email ID	rushikesh.madas@wipro.com
Problem Title	**Drone Fleet Management System**
Domain	Aerospace

Problem Description

Build a Drone Fleet Management System in Python (Aerospace domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Aerospace domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.

Problem Statement #44

Participant Name	ALAHARI VARSHA
Email ID	alahari.varsha@wipro.com
Problem Title	**Cybersecurity Incident Response Tracker**
Domain	Security

Problem Description

Build a Cybersecurity Incident Response Tracker in Python (Security domain) integrating ALL the following programming concepts:

Variables, Data Types & Literals:

- Use integer (including hex/binary/octal literals), float, string, boolean, and complex data types relevant to the Security domain.
- Demonstrate underscore in numeric literals for readability.

Base Conversions & Type Conversions:

- Convert primary identifier fields to binary, octal, and hexadecimal representations.
- Perform explicit int(), float(), str(), and bool() conversions as appropriate in the system.

Arithmetic & Bitwise Operators:

- Perform meaningful arithmetic calculations (totals, averages, rates, percentages).
- Use at least three bitwise operators (&, |, ^, <<, >>) for flag manipulation or status encoding.

Logical & Comparison Operators:

- Apply compound logical conditions (and, or, not) for validation and business rules.
- Use all six comparison operators in sorting, filtering, and condition checking.

Conditional Statements:

- Use simple if, two-way if-else, and multi-way if-elif-else constructs for domain-specific decision trees.
- Include nested conditions where applicable.

Lists (All Functions & List Comprehension):

- Maintain at least two lists; demonstrate: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend(), copy(), clear(), and min(), max(), sum().
- Write at least two list comprehensions (one with condition, one nested).
- Demonstrate use of sorted() and enumerate() with lists.

Tuples:

- Use at least two tuples for immutable domain records.
- Demonstrate tuple unpacking, slicing, count(), and index() methods.

Sets:

- Use sets for duplicate-free collections; demonstrate add(), discard(), union(), intersection(), difference(), symmetric_difference(), issubset(), issuperset(), and frozenset().

Dictionaries:

- Use at least two dictionaries; demonstrate get(), setdefault(), update(), pop(), popitem(), keys(), values(), items(), and clear().
- Write at least one dictionary comprehension.

Loops & Control Flow:

- Use for loop with range, enumerate, and zip.
- Use while loop with a validation or simulation use case.
- Demonstrate break (early termination), continue (skip condition), and pass (stub function body).

match-case Statement:

- Implement a menu or status routing system using Python's structural pattern matching (match-case) with at least 5 cases including a default wildcard.

String Methods (Comprehensive):

- Demonstrate all of: upper(), lower(), title(), capitalize(), strip(), lstrip(), rstrip(), replace(), split(), join(), find(), rfind(), startswith(), endswith(), format(), f-strings, zfill(), center(), ljust(), rjust(), count(), encode(), maketrans(), translate(), isdigit(), isalpha(), isalnum().

Functions (Modular Approach):

  - a) At least one function using POSITIONAL-ONLY parameters (/) for core computation.
  - b) At least one function using KEYWORD-ONLY parameters (*) for configuration.
  - c) At least one function using *args and **kwargs alongside regular parameters (mixed).
  - d) Demonstrate default parameter values.
  - e) Implement exception handling with try/except blocks:
- Catch built-in exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, IndexError.
- Define and raise at least ONE custom exception class inheriting from Exception.
- Use else and finally clauses in at least one try block.

File Handling:

- Write structured data to at least one .txt file using with statement.
- Read the file back and display contents.
- Handle FileNotFoundError and IOError gracefully.
- Demonstrate append mode ('a') and read mode ('r').

Iterators:

- Implement a custom iterator class with __iter__() and __next__() methods relevant to the domain.
- Demonstrate StopIteration.

Generators:

- Write at least one generator function using yield for lazy evaluation.
- Demonstrate generator expression syntax as well.

Object-Oriented Programming:

  - a) Define at least a two-level inheritance hierarchy (Base → Derived) with a third level if applicable.
  - b) Implement: __init__, __str__, __repr__, __len__ or __eq__ operator overloading.
  - c) Encapsulation: at least one private attribute (__attr) with @property getter and setter.

- d) @classmethod for alternative constructors.
- e) @staticmethod for utility functions.
- f) Method overriding and polymorphism: the same method name behaves differently across subclasses.
- g) Demonstrate isinstance() and issubclass() checks.

Stacks:

- a) Array-Based Stack: Implement a Stack class using a Python list with push(), pop(), peek(), is_empty(), and size() methods. Use it to implement an undo/redo feature relevant to the system.
- b) Linked-List-Based Stack: Implement a Node class and a LinkedStack class with the same interface. Use it to maintain a history or audit log in the system.

- For both stacks, demonstrate handling of stack underflow (popping from empty stack) with appropriate exception.